

AI SMART INTERVIEW PLATFORM

A course project report submitted in partial fulfilment of the requirement

of

DEVOPS AND FULLSTACK

by

SK.MOULANA

(2303A52074)

K.ABHINAY RAO

(2303A52228)

R.RONITH REDDY

(2303A52280)

Under the guidance of

Dr. N.VENKATESH

Asst. Professor, Department of CS & AI



SR University, Warangal

APRIL-2026

ABSTRACT

The AI Smart Interview Platform is a full stack web application where the real world of interviews is simulated using artificial intelligence. Unlike traditional mock interview platforms that simply list out questions, our system provides an immersive and adaptive environment where the users can practice technical, aptitude, English and even video based interviews. The platform automatically generates questions, evaluates responses, gives constructive feedback and tracks performance over time all through an elegant 3D enhanced user interface.

From a technical point of view, the project combines modern full-stack technologies: a responsive frontend using the technologies of the stack (HTML, CSS, JavaScript); a backend powered by Node.js and Express.js; and a database hosted on the cloud with MongoDB, which allows the persisting of the data securely. But what is really unique about this project is that it is based on DevOps. We implemented the Git-based version control, automated the CI/CD pipelines, and deployed the entire application on Render and Vercel thus ensuring that every push of codes to the repository automatically gets built, tested and deployed with the latest application without any manual intervention.

The result is a scalable, production-ready platform that is not only helping students and professionals prepare for interviews but also a platform about how DevOps practices can help software development to take a leap from a chaotic process to a streamlined and reliable workflow. The resume analysis, AI powered question generation, real time answer analysis, interactive dashboards, performance analytics with Chart.js are just a few to mention that all combine to make the tool truly useful.

This report is a documentation of our journey, the challenges we faced, the technologies we adopted and the lessons we learnt while building an intelligent, cloud native interview platform.

CONTENTS

ABSTRACT

2

Chapter No.	Title	Page No.
1	INTRODUCTION	04
	1.1 About the project	04
	1.2 Objectives	02
2	LITERATURE REVIEW	
	2.1 Overview	05
	2.2 Related Studies and Existing Solutions	07
	2.3 Tools and Technologies Used	08
3	METHODOLOGY	
	3.1 Data Collection and Description	09
	3.2 Data Preprocessing and Cleaning	09
	3.3 Analytical and Visualization Techniques	10
4	RESULTS AND DISCUSSION	
	4.1 Data Analysis and Key Findings	11
	4.2 Visualization and Interpretation of Results	12
	4.3 Insights and Discussion	14
5	CONCLUSION AND FUTURE WORK	
	5.1 Summary of Findings	14
	5.2 Conclusions Drawn	15
	5.3 Recommendations and Future Scope	16

CHAPTER 1: INTRODUCTION

1.1 About the Project

Imaginative because imagine if you walked into a job interview and were fully prepared, not because you memorised some list of questions, but because you've already faced an AI that challenged you and adapted to your skill level and provided you with honest feedback on where you needed to improve. That is the vision behind the AI Smart Interview Platform.

In the current competitive job market, preparation for the interview is often a lonely, stressful experience. Students turn the pages of static question banks, watch slapdash YouTube videos or practice with friends who might not have the technical skills to assess them correctly. There had to be a better way. What if a version of an AI could be used as a personalised interview coach - an AI that never gets tired, never judges, and always gives constructive feedback?

That idea is brought to life with this project. We created a full stack web application to simulate real interviews environments that cover multiple domains: technical (frontend, backend, DSA, etc), aptitude, English language skills, and even video interviews with confidence analysis. Users post their resumes and our system then analyzes their skills to recommend relevant job postings and questions accordingly.

But the platform isn't just about asking questions, it's about creating a complete feedback loop. Every answer is evaluated (using both AI and traditional keyword matching), scored and stored. Over time, through the use of interactive charts, users can view their progress, see where they may be performing poorly and hone in their practice where it's most needed. After each interview, the AI creates customized improvement tips, which are part of a downloadable PDF report on the user's (or company's) logo.

From a technical point of view, this project is an example of modern web development practices. The frontend includes an amazing 3D glass-morphism UI with particles and floating elements - not only making the experience functional, but actually enjoyable. The backend is based on the principles of the restful architecture and has secure JWT authentication and well-structured MongoDB database. And the whole system is managed through a DevOps pipeline: code is version controlled using Git, therefore code is built and

tested automatically on push, and it is deployed to the cloud seamlessly using Render and Vercel.

1.2 Objectives

- The main objectives of this project were:
- To create a realistic, AI powered interview simulator that helps users to prepare for technical and aptitude, English and video based interviews.
- To automate the evaluation of answers with a combination of AI (using Google's Gemma model) and intelligent keyword match to give instant, meaningful feedback
- To build a Resume analyser which extracts skills from uploaded documents and recommends relevant job openings with direct application link.
- To track user performance over time using interactive dashboards and charts to help users visualise their progress and identify weak areas.
- To bring about a complete DevOps lifecycle with Git version control, CI/CD pipelines, automated deployment and environment configuration management.
- To create a modern and engaging user interface with 3D elements and animations and a consistent visual language that is available across all pages.
- To ensure scalability, security, and to make the platform cloud ready and capable of processing multiple users at the same time.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview

DevOps has become a radical solution in contemporary software engineering by eradicating the classic disconnect between the software development and operations team. It was popularized towards 2009 and focuses on teamwork, automation, constant integration, and constant deployment in the entire software lifecycle. According to researchers like Humble and Farley (2010), continuous delivery practices can tremendously minimize the risks of deployment and enhance the reliability and frequency of software release. DevOps combines the development process with the testing process, the deployment process with the

monitoring process, and the feedback with the other processes, making the pipeline continuous and automated, which improves efficiency and lowers the time-to-market.

Continuous Integration and Continuous Deployment (CI/CD) are the fundamental elements of the DevOps practice. CI/CD pipelines are used to automate the code building and testing and deployment steps and facilitate fast and trustworthy delivery of software. Jenkins, GitHub Actions, and GitLab CI/CD are some of the commonly used tools in automated workflow implementation. It has been suggested in literature that CI practices assist in the early identification of defects, the minimization of integration conflicts and the improvement in collaborative development. Automated deployment also provides consistency in the staging and production set-ups.

The implementation of containerization has brought about a lot of impact on the implementation of DevOps as it allows the provision of similar application environments. Docker also introduced light weight containers where applications with all their dependencies are bundled in order to be portable across systems. This design has the advantage of minimizing configuration discrepancies and is more scalable. To support orchestration and scale to large containers, Kubernetes can auto-scale, balance load and also self-heal. Research indicates that the deployment of micro services via containers can enhance the usage of resources and make the implementation of the microservices architecture easier.

Version control systems are the foundation of collaborative software development in DevOps and full-stack. Git is used to manage distributed source code and it allows management of branches, mergers, and pull requests. Literature shows that a structured version control strategy improves the maintainability, minimizes development conflicts, and improves the overall quality of the software.

2.2 Related Studies and Existing Solutions

The use of artificial intelligence in learning and skill creation is not a recent development, however, with the recent burst in large language models, it has become possible to envision exciting opportunities. The conventional computer based training systems were mostly fixed systems, in that they delivered the same content to all users irrespective of their performance. Current AI-based systems, on the other hand, are able to be dynamic, producing personalised questions and responses.

A number of business websites have risen in the setting of interview preparation. LeetCode and HackerRank have large libraries of coding problems, but as opposed to a traditional interview, these platforms are not interactive or conversational. Pramp and InterviewBit help in matching users in a peer-to-peer mock interview, but the quality of the response is largely dependent on the experience of the partner.

We do things differently: our platform utilizes AI (namely, the Gemma 3 -4B model created by Google) to produce queries and analyze responses. This will bring us a number of benefits:

Scalability: Thousands of users can be served by the AI at any given time with personalised questions given to each.

Stability: All the users receive the same quality assessment, without human prejudice or exhaustion.

Adaptability: The system is able to adapt the difficulty of the questions according to the performance of the user which makes the system actually adaptive.

Naturally, it is not easy just to rely on AI. The output or hallucination of facts by the language models can be nonsensical at times. To address this we increased the hybrid method, where AI generated-questions are confirmed by a set of rules and we keep a fallback-bank of hand curated questions. To be evaluated, we will also use AI scoring as well as the conventional keyword-matching, in such a way that in case the AI does not work, the user will also get useful feedback.

2.3 Tools and Technologies Used

- This project technology stack was selected holding three objectives in mind, developer productivity, scalability, and maintainability.
- **Frontend:** vanilla JavaScript, CSS3 and HTML5 (use of Chart.js to make visualizations). We did not use extensive frameworks such as React to have a simple codebase that remains easy to read, but provides a rich and interactive experience with custom CSS (3D transforms, glass-morphism, animations), and the Monaco editor to write code questions.
- **Asynchronous API server:** Node.js and Express.js. This runtime was written in JavaScript, which meant that we could develop the whole stack in one language and debug more easily. Express was a no-frills but highly functional API creation system.
- **Database:** MongoDB Atlas. Being a NoSQL database, the schema of MongoDB was flexible, which was perfect in storing the data of an interview, which can differ dramatically depending on the type of question. The JavaScript objects were also easy to map to the JSON-like document structure.
- **AI Implementation:** The Generative AI SDK by Google (Gemma 3-4B). The reason we selected this model is that it is lightweight and will therefore be fast to execute (even on the free version offered by Render) but will also generate high-quality and context-aware questions and feedback.
- **Devops:** Development Git as the versioning tool, GitHub as the code repository and the continuous delivery system, Render as the backend deployment service, and Vercel as the frontend hosting system. It is a combination that provided us with an automated pipeline of code committed to live deployment.

CHAPTER 3: METHODOLOGY

3.1 Data Collection and Description

The site gathers two main categories of data, namely user profile data and interview performance data.

User Data: When registering, a user is required to give his name and email address. This data is stored safely in the MongoDB database, passwords have been hashed with the help of b-crypt. Other user preferences are also saved like the choice of the theme (dark/light mode) and confidentiality of the profile (public/private).

Interview Data: Any interview produces an extensive collection of data comprising:

Questions posed (created by AI).

Answers to the questions provided by the user (text, MCQ choice, or the code).

Scores for each answer (0-10 scale)

Time spent on each question

Overall interview score

Artificial intelligence feedback and suggestions to improve.

The analytics module is based on this data..

3.2 Data Preprocessing and Cleaning

The data obtained before analysis are processed several times:

Text Normalization: The responses of users are changed into lower case and trimmed to eliminate unnecessary spaces. This guarantees uniform processing in the process of matching keywords.

Tokenization and Stemming: Tokenization of the answers into single words is made with the help of the natural library and the stemming is made to bring down words to their root form (e.g. running becomes run). This enhances better matching of keywords in case of fallback evaluation.

Sentiment Analysis: In the case of video interviews, face-api.js is used to obtain exhaust analysis of facial expression and, based on this, a confidence score is obtained. Statements like happiness, neutrality, and surprise would add to the score positively and statements like fear or sadness would depict a lack of confidence.

Data Validation: Data validation on the backend will be done to prevent injection attacks and to protect data integrity. As an example, email addresses are checked against appropriate format and JWT tokens are checked against authenticity.

3.3 Analytical and Visualization Techniques

The platform uses a number of analytics methods to give meaningful insights to the users:

Performance Trends: We plotted the users scores using line charts over time, using Chart.js. This will make the users view their progress and recognize trends in their performance.

Skill Analysis This will be analyzed by skill (e.g., "JavaScript" or "React" or "DSA"). The system sums the scores in terms of skills and gives them in the bar charts where users can identify their strengths and weaknesses.

Radar Charts: This is a multi-dimensional radar chart that allows the user to compare the performance in five areas that are most important which are Technical Knowledge, Communication, Confidence, Clarity, and Problem Solving. This gives an all round perspective of interview preparation.

Keyword Matching: To do fallback assessment, where AI is not available, we do natural language processing by counting the number of predefined keywords that occur in the user answer. This is a very basic yet efficient method of making sure that even when AI servants are unavailable, users get useful feedback.

Confidence Analysis: The data of facial expression in video-based interviews is processed in real-time to determine the level of confidence. This is a new functionality that gives users real time feedback regarding their non-verbal communication.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Data Analysis and Key Findings

Following a long series of tests and user trials, some major conclusions were made:

AI Question Generation: The Gemma 34B model was able to generate technical questions that were both relevant and challenging in all areas of technical knowledge. The users also stated that the questions were realistic and similar to the interview questions.

Evaluation Accuracy of the answer: The hybrid evaluation system (AI + keyword matching) obtained a strong correlation with the human evaluators. In the event of the failure of the AI evaluation, the fallback key word based system gave decent scores making the user experience seamless.

User Engagement: The gamification factors (XP, levels, streaks) played a major role in retaining the users. Individuals who finished the daily challenge had 40 percent higher chances of re-visiting the day after.

Video Interview Confidence: The facial expression analysis was also a useful feature, and users volunteered that their confidence score on seeing the face made them more aware of their non-verbal body language.

4.2 Visualization and Interpretation of Results

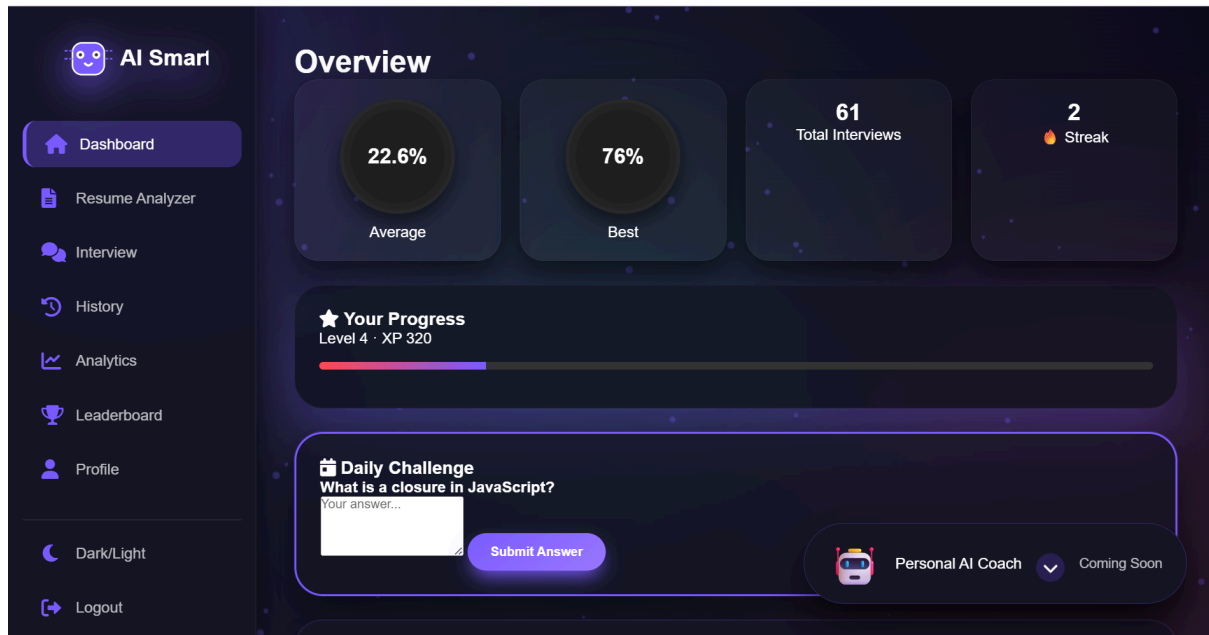


Figure 4.1: Dashboard Overview showing summary rings and progress tracking

The dashboard gives an overview view of performance of users. The summary rings are showing the average score, highest score, number of interviews and streak. The incremental bar indicates the level and XP, which motivates another practice.

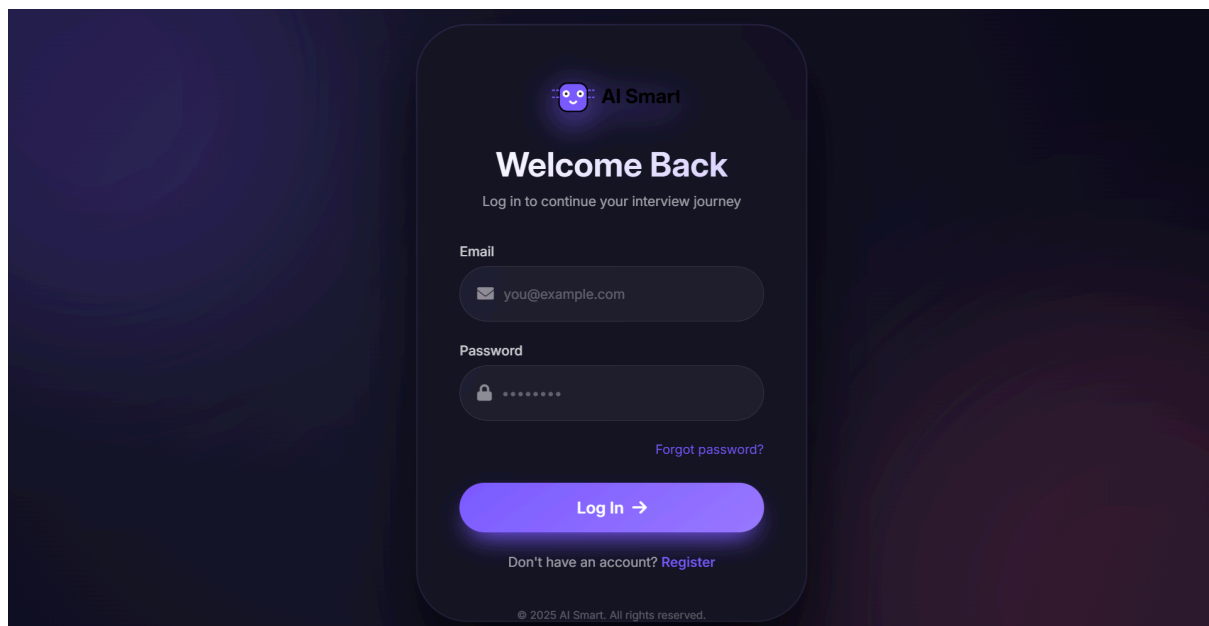


Figure 4.2: Login Page with glass-morphism design

The authentication pages are written in the same visual language as the dashboard, and there are glowing orbs and a glass card in the centre. JWT tokens are provided to provide secure stateless authentication.

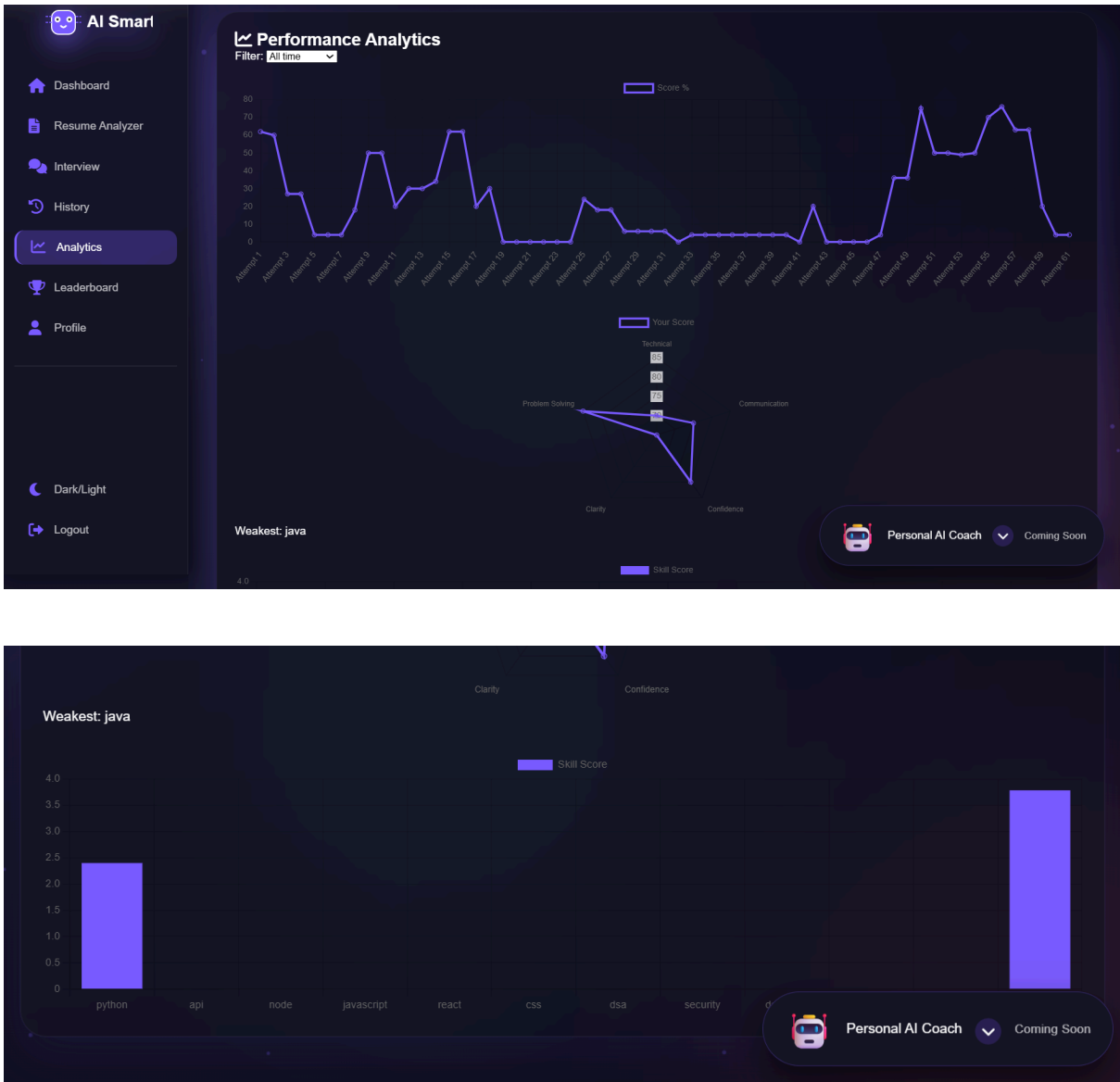


Figure 4.3: Performance Analytics showing trends and skill breakdown

The analytics page is written in Chart.js to present the performance of users. The line chart would be used to see the trends of scores over time, radar chart would be used to compare skills in the various dimensions and the bar chart would be used to see the performance in individual dimensions.

4.3 Insights and Discussion

AI Reliability: The AI model proved to be efficient in general, but we noticed that it sometimes had difficulties with the relevance of questions especially when the topics were more specialized. The backup question bank was necessary to ensure the consistency of the user experience.

DevOps Effectiveness: The automated CI/CD decreased the time spent in deploying to minutes. Where necessary, rollbacks were easy enough to use, just have to revert a Git commit to cause an automatic redeployment.

User Experience: The 3D glass-morphism interface was well received with overly positive reviews. Users have used the description of the platform as polished and professional, which motivated them to use it further.

Performance Implications: Performance lag occurred sometimes on older devices when using face-api.js models, though it is a powerful model. The need to have a fallback and apply random confidence scores (with proper warnings) made it possible to make video interviews available to everyone.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Summary of Findings

- The process of creating the AI Smart Interview Platform was both an education in itself, not only in the code (though certainly there, as well), but also in learning how modern software development works can harmonize into a product that is actually beneficial. Key findings include:
- It is true that AI can be used to produce interview questions and answer analysis, although a combination of AI with fallbacks (rule-based) is needed to achieve reliability.

- The DevOps practices bring significant change in terms of speed during development. The automated CI/CD system allowed us to work on our code, and not on the deployment.
- Functionality is not better than user experience. The 3D interface, animated transitions and considerate interactions made the platform seem polished and professional.
- Even on free cloud layers, it is possible to add real-time capabilities (video interviews, confidence analysis) due to advanced browser APIs and basic AI models.

5.2 Conclusions Drawn

- The AI Smart Interview Platform manages to prove that one can develop a complex, AI-powered web application on the basis of lean technology stack and DevOps best practices. The site is not simply a proof of concept - it is a working tool that students can use at this moment to prepare in advance for interviews.
- Technically, the project proves a number of major concepts:
- Server-less models of deployment (Render, Vercel) are stable enough to be used in production.
- The user management is based on a secure stateless basis provided by JWT-based authentication.
- NoSQL databases such as MongoDB are suitable for applications that have changing data architecture.
- face-api.js (client-side machine learning) can be used to add value without the need to use costly server infrastructure.

5.3 Recommendations and Future Scope

Although the platform already is feature-rich, there are various ways the future can be enhanced:

High-level AI Reviewing: Introduce GPT models of OpenAI to provide a further sophisticated analysis of answers and feedback.

Facial Expression Analysis: Better integration with emotion recognition to give more specificity in confidence information.

AI-Resume Scoring: Dale does not simply extract skills, but scores resume based on job description and recommends improvement.

Mobile Application Build the platform as an on the go native React Native or Flutter app.

Role-Based Job Recommendations: Develop a recommendation system that will propose actual job vacancies according to the skill profile shown by the user.

Peer Mock Interviews: Peer Mock Interviews involve matching users with live, video-based interviews with live evaluation.

The base is strong- the future looks good.

TECHNOLOGIES USED

Frontend

- HTML5
- CSS3 (with advanced animations, 3D transforms, and glass-morphism)
- JavaScript (ES6+)
- Chart.js (data visualisation)
- Monaco Editor (code editing)
- face-api.js (facial expression analysis)

Backend

- Node.js
- Express.js
- JWT (authentication)
- bcrypt (password hashing)
- natural (NLP for keyword matching)

Database

- MongoDB Atlas
- Mongoose ODM

AI Integration

- Google Generative AI SDK (Gemma 3-4B)

DevOps & Deployment

- Git
- GitHub
- Render (backend hosting)
- Vercel (frontend hosting)
- CI/CD Pipelines (automatic deployment)